Exp 6

Aim

To create and compile a smart contract using Solidity, deploy it on the Ethereum Sepolia testnet through Remix IDE and MetaMask, and execute transactions to store and update data on the blockchain.

Theory

A smart contract is a self-executing program deployed on a blockchain. It contains functions and variables that define how data is stored, accessed, and modified. Smart contracts on Ethereum are written using Solidity, a high-level programming language designed for developing decentralized applications.

Remix IDE is a browser-based development environment used to write, compile, deploy, and test Solidity smart contracts. The Solidity compiler converts source code into bytecode and an Application Binary Interface (ABI), which are required for deployment and interaction with the contract.

MetaMask is a cryptocurrency wallet that connects Remix IDE to the Ethereum network and allows users to approve blockchain transactions. Sepolia is an Ethereum testnet used for testing smart contracts without spending real ETH. Test ETH is used to pay transaction gas fees.

In this experiment, a `SimpleStorage` contract was created with a variable to store an unsigned integer. The `setValue()` function modifies the stored value and generates a blockchain transaction, while the `getValue()` function reads the current value without modifying the blockchain state. An event named `ValueChanged` records the updated value in the transaction logs.

Procedure

1. Opened Remix IDE and created the file `SimpleStorage.sol`.
2. Wrote the Solidity smart contract containing `setValue()` and `getValue()` functions.
3. Compiled the contract using a compatible Solidity compiler version.
4. Connected Remix IDE to MetaMask and selected the Sepolia testnet.
5. Deployed the compiled smart contract and confirmed the deployment transaction.
6. Executed `setValue(100)` and verified the stored value using `getValue()`.
7. Executed `setValue(250)` and verified that the stored value was updated to 250.
8. Verified the transaction details and confirmation status using Sepolia Etherscan.

Output

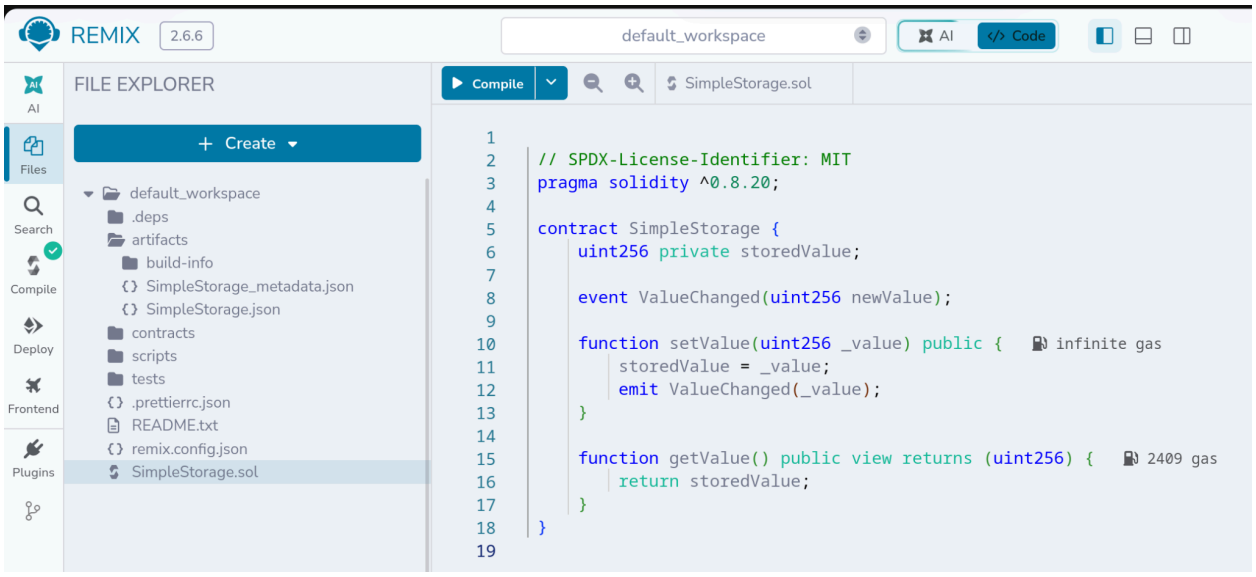
The smart contract was successfully compiled and deployed on the Sepolia testnet.

Operation	Expected result	Observed result
`setValue(100)`	Value stored as 100	Successful
`getValue()`	100	100
`setValue(250)`	Value updated to 250	Successful
`getValue()`	250	250

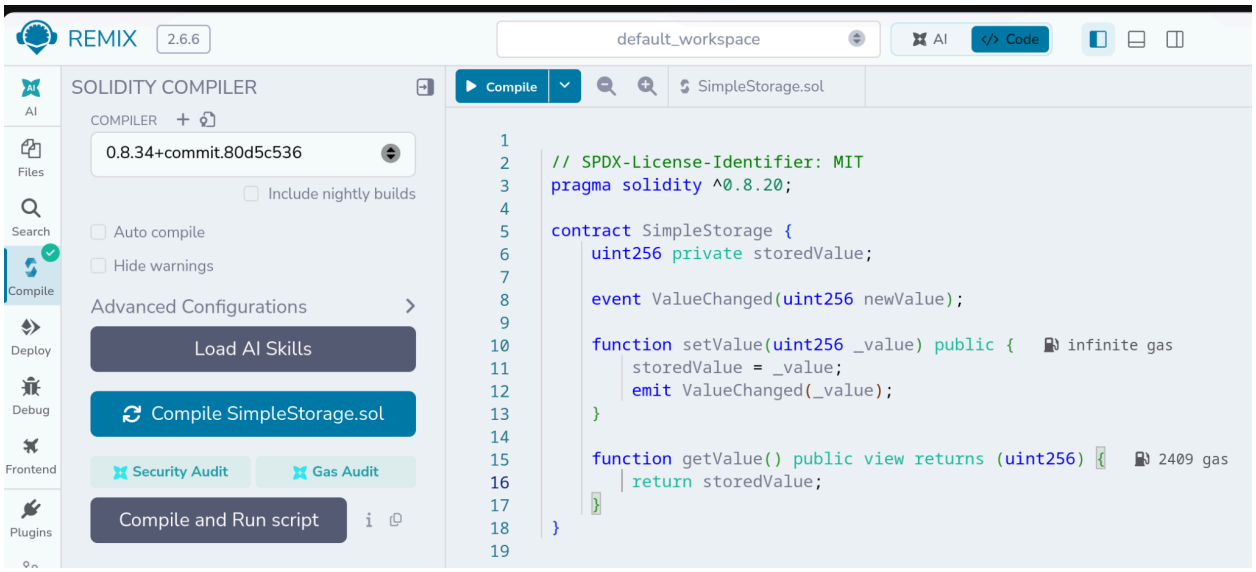
The transactions were confirmed, and the final stored value was verified as `250`.

Output:

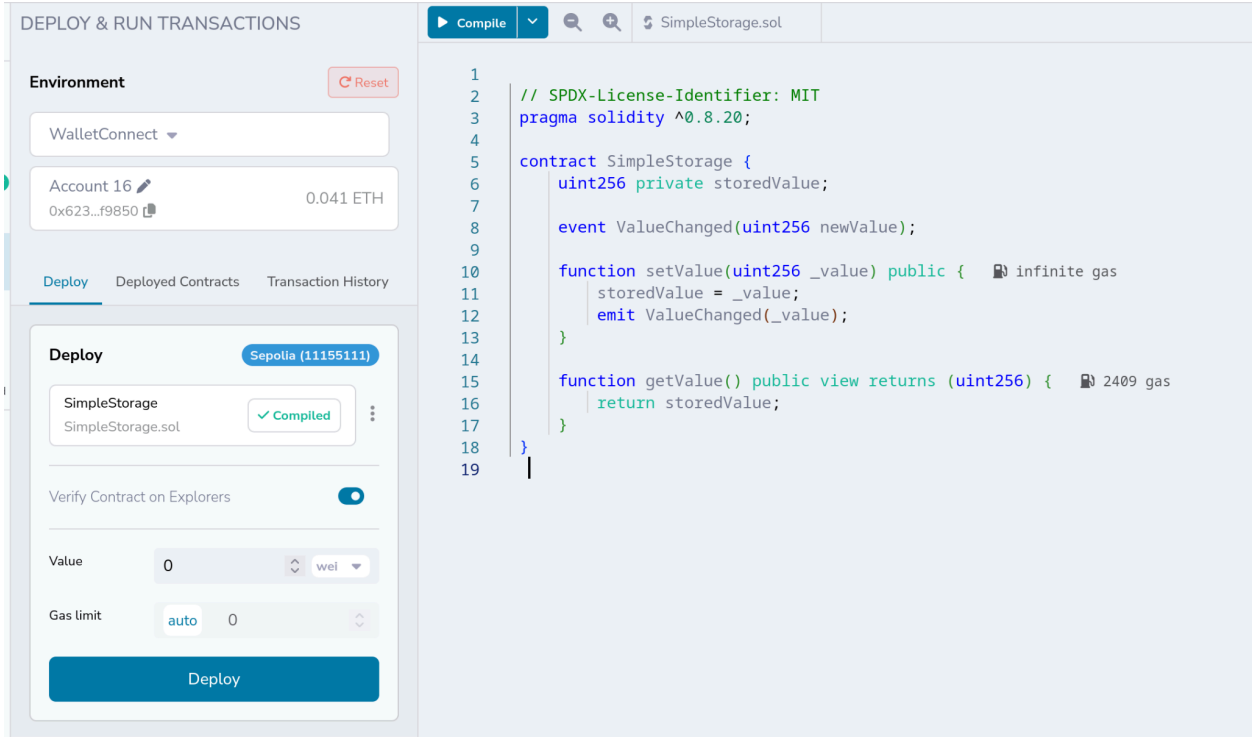
1. Solidity source code in Remix IDE.



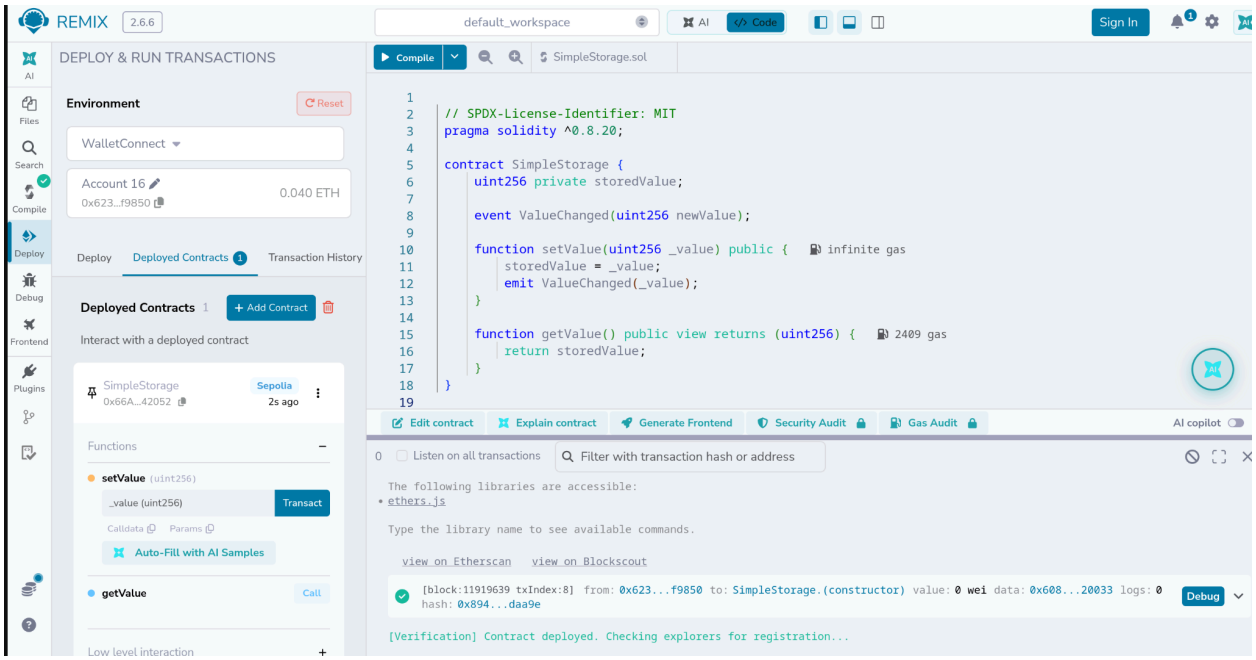
2. Successful compilation.



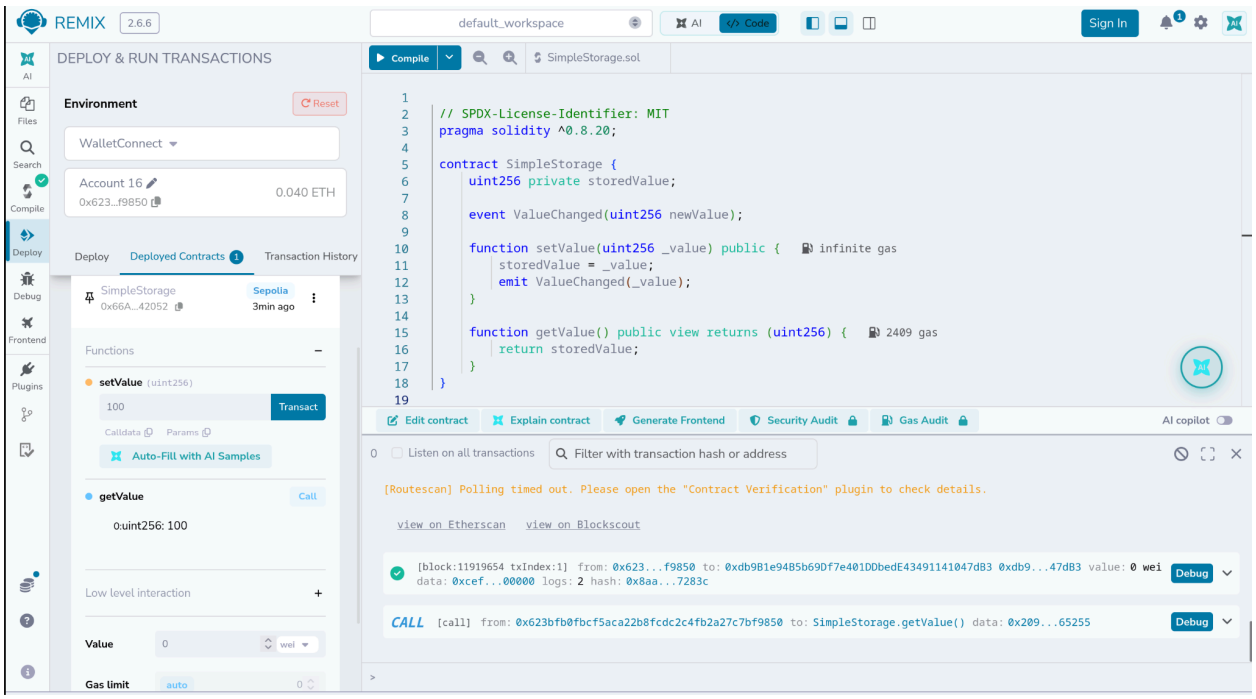
3. Remix connected to MetaMask on Sepolia.



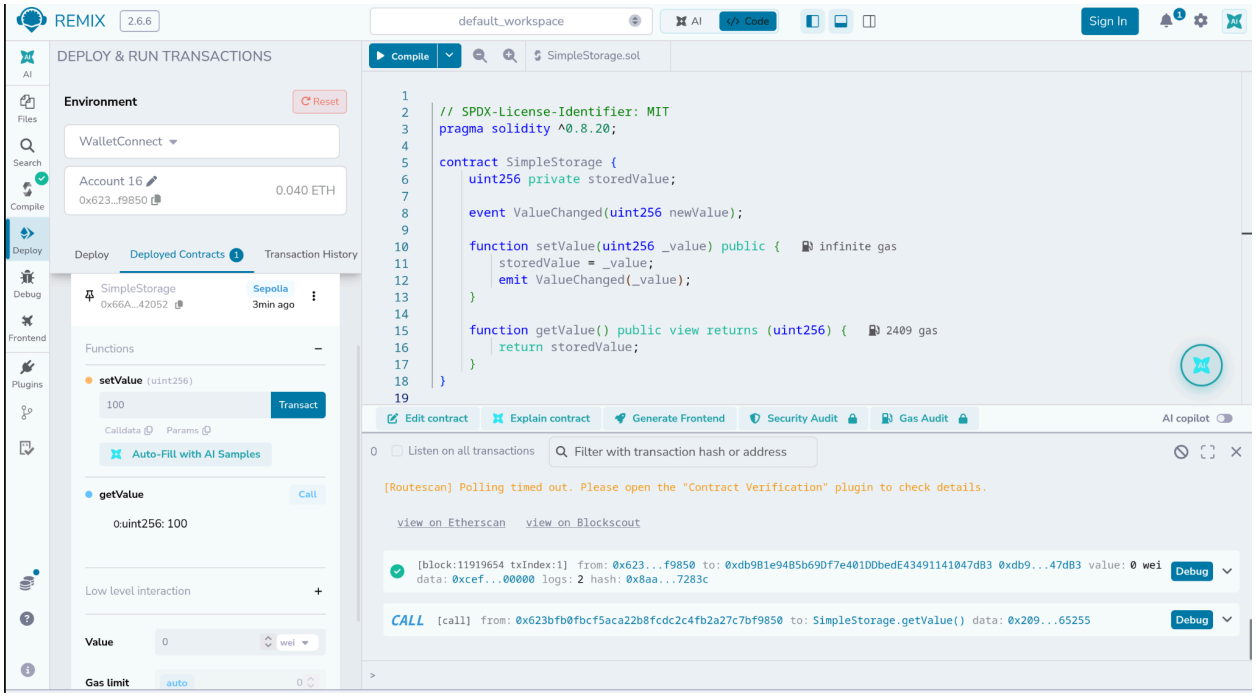
4. Successful contract deployment.



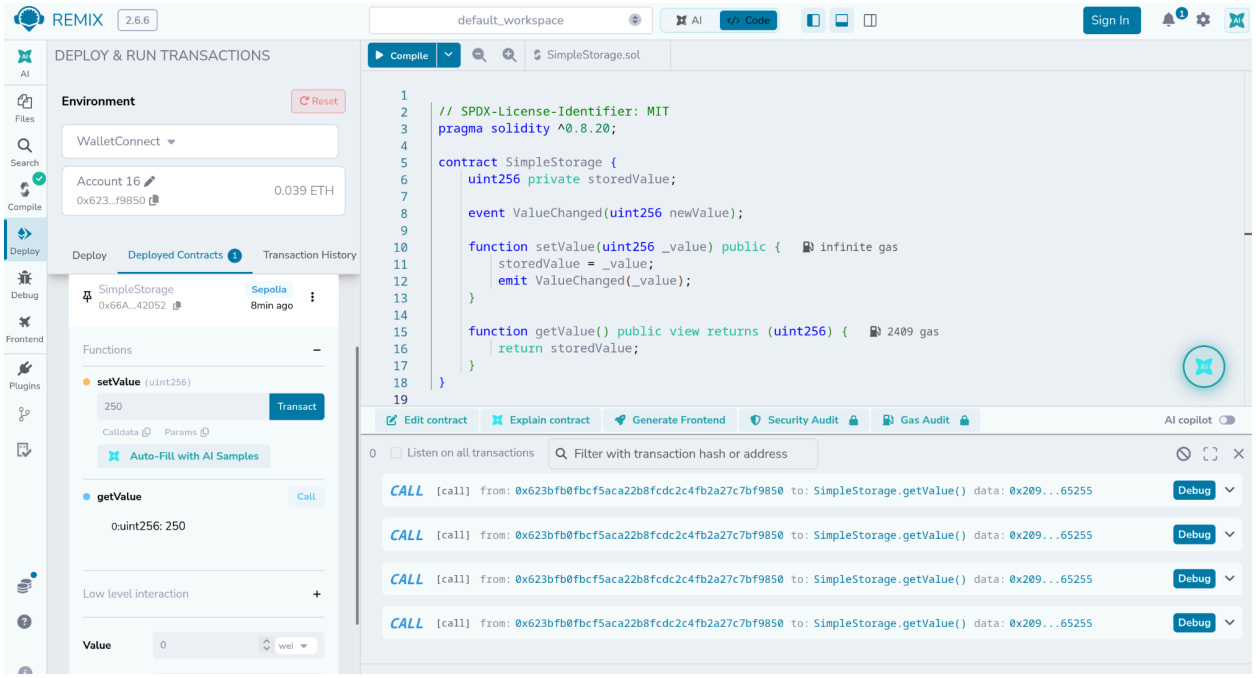
5. First transaction, `setValue(100)`.



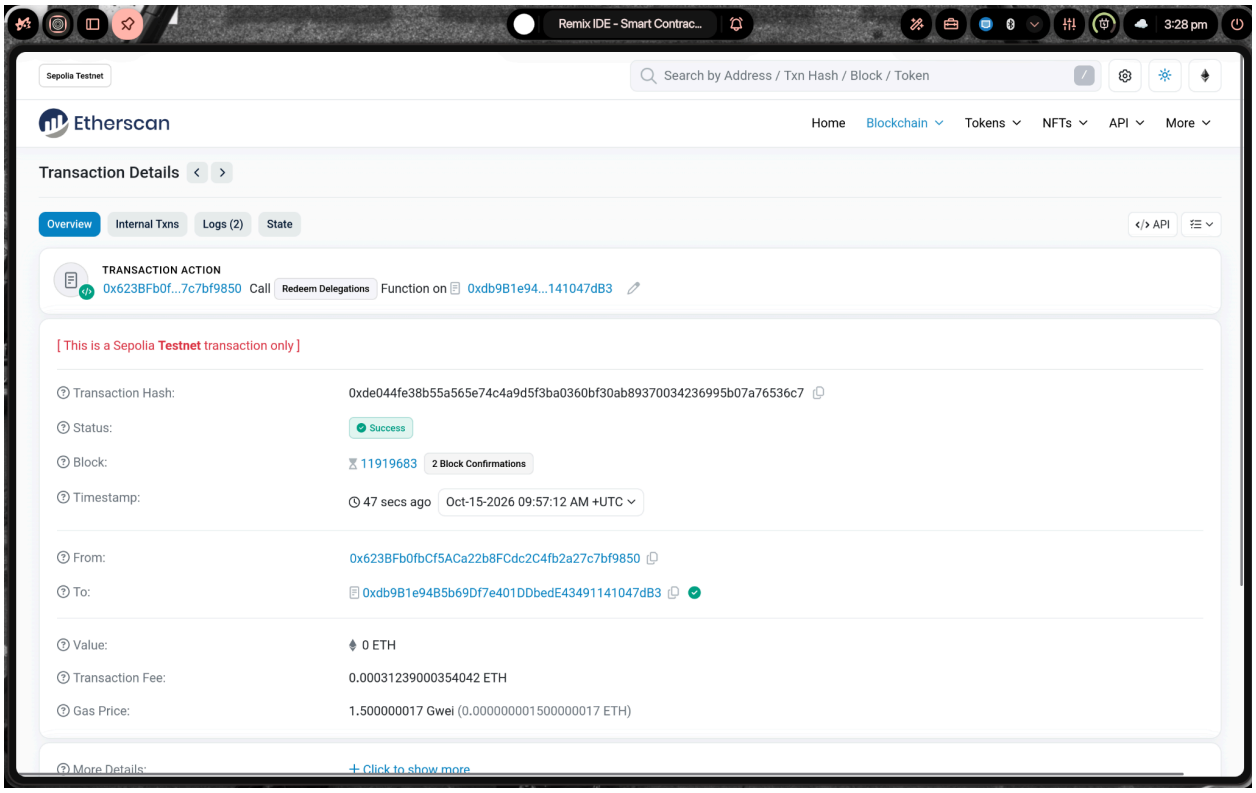
6. `getValue()` returning 100.



7. Second transaction, `setValue(250)`, and final verification returning 250.



8. Sepolia Etherscan transaction verification, if captured.



Conclusion

The smart contract was successfully created using Solidity, compiled in Remix IDE, and deployed on the Ethereum Sepolia testnet using MetaMask. State-changing transactions were executed to store and update integer values on the blockchain. The results were verified using the contract's read function, demonstrating the process of smart contract deployment, transaction execution, and state verification on Ethereum.